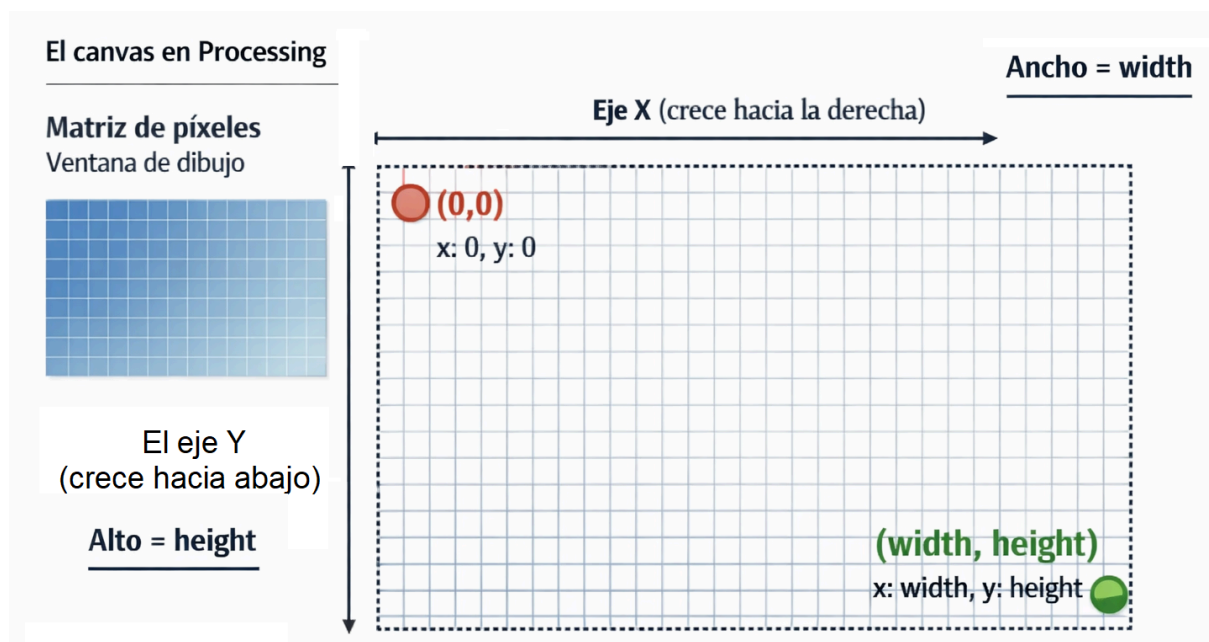


En Processing, el **canvas** es la ventana donde se dibuja todo lo que produce el programa. Puede pensarse como una superficie rectangular compuesta por píxeles, en la cual se ubican las formas, imágenes y textos que generamos mediante código. Cada punto dentro de esa superficie puede describirse mediante un sistema de coordenadas que indica su posición horizontal y vertical.

El sistema de coordenadas de Processing funciona de manera similar a un plano cartesiano, pero con una diferencia importante en su orientación. El punto de origen se encuentra en la esquina superior izquierda de la pantalla y corresponde a la coordenada **(0,0)**. A partir de allí, el eje **x** crece hacia la derecha y el eje **y** crece hacia abajo. Esto significa que cuando aumentamos el valor de **x** nos movemos horizontalmente hacia la derecha, y cuando aumentamos el valor de **y** nos desplazamos verticalmente hacia la parte inferior del canvas. Esta orientación es distinta a la del plano cartesiano matemático tradicional, donde el origen suele ubicarse en el centro y el eje vertical crece hacia arriba.



El tamaño del canvas se define al inicio del programa mediante la función **size()**, que establece el ancho y el alto de la ventana en píxeles. Por ejemplo, **size(800,600)** crea una ventana de 800 píxeles de ancho por 600 de alto. La elección de estas dimensiones no es solo un aspecto técnico, sino también una decisión compositiva, ya que determina el espacio disponible para organizar los elementos visuales y la relación entre alto y ancho (16:9, 4:3, etc.).

Processing ofrece además dos variables automáticas que permiten conocer en cualquier momento las dimensiones del canvas: **width** y **height**. La variable **width** contiene el ancho total de la ventana, mientras que **height** contiene su altura. Estas variables se

actualizan automáticamente a partir del tamaño definido en `size()` y pueden utilizarse dentro del programa para calcular posiciones de manera flexible. Por ejemplo, si queremos ubicar un elemento en el centro del canvas, podemos usar `width/2` y `height/2` en lugar de escribir coordenadas fijas.

El uso de `width` y `height` resulta especialmente útil porque permite que el programa se adapte fácilmente a diferentes tamaños de canvas. En lugar de depender de valores absolutos, las posiciones pueden calcularse en relación con las dimensiones de la ventana, lo que hace que la composición visual mantenga su estructura incluso si se modifica el tamaño del espacio de dibujo.

Por eso a partir de ahora y para siempre, en cada ejercicio de la cursada, buscaremos -en lugar de indicar posiciones mediante valores fijos de píxeles- calcular las coordenadas en relación con el tamaño del canvas utilizando las variables `width` y `height`.

Trabajar con fracciones de estas variables permite definir posiciones que se adaptan automáticamente a distintos tamaños de ventana. Por ejemplo, si se desea ubicar un elemento exactamente en el centro de la pantalla, se pueden utilizar las coordenadas `width/2` y `height/2`. De esta manera, sin importar cuál sea el tamaño del canvas, el objeto siempre aparecerá en el centro.

Este mismo principio puede aplicarse para ubicar elementos en otras zonas del espacio de dibujo. Por ejemplo, `width*0.25` ubica un punto a un cuarto del ancho de la pantalla, mientras que `height*0.75` corresponde a tres cuartos de su altura. Estas proporciones permiten organizar la composición visual utilizando relaciones espaciales en lugar de posiciones rígidas.

El posicionamiento relativo también facilita trabajar con los bordes y los márgenes del canvas. Por ejemplo, si se quiere mantener un margen constante respecto al tamaño de la ventana, se puede definir una distancia proporcional, como `width*0.05` o `height*0.05`. Esto permite que el espacio alrededor de los elementos se mantenga equilibrado incluso cuando cambian las dimensiones del canvas.

Este enfoque forma parte de una estrategia más amplia que puede llamarse **diseño adaptable**, el cual es muy útil cuando pensamos las múltiples pantallas o soportes (impresoras 2D, 3D, proyecciones) que tendrá nuestro desarrollo.

Para organizar la ubicación de los elementos en pantalla también resulta útil utilizar algunas **estrategias prácticas de layout**. Una de las más simples consiste en crear **variables intermedias** que representen posiciones o medidas importantes dentro del canvas. En lugar de escribir muchas veces expresiones como `width*0.5` o `height*0.25`, se pueden definir variables que describan esas relaciones de forma más clara.

Por ejemplo, se pueden declarar variables como:

```
float centroX = width/2;  
float centroY = height/2;  
float margenX = width*0.1;  
float margenY = height*0.1;
```

Estas variables permiten organizar el código de manera más legible y facilitan la modificación de la composición. Si en algún momento se desea cambiar el tamaño de los márgenes o mover la estructura general del dibujo, basta con modificar el valor de esas variables en lugar de buscar y cambiar múltiples coordenadas en el programa.

Otra estrategia frecuente consiste en **construir grillas simples** utilizando `width` y `height`. Una grilla permite dividir el canvas en secciones proporcionales que funcionan como guía para ubicar elementos. Por ejemplo, si se quiere dividir la pantalla en cuatro columnas iguales, se puede calcular el ancho de cada columna como `width/4`. De la misma manera, si se desea dividir la altura en tres filas, cada una puede tener un tamaño de `height/3`. A partir de estas divisiones es posible ubicar objetos en posiciones regulares dentro de la estructura de la grilla.

Posiciones Relativas y absolutas.

Al comenzar a trabajar con estas técnicas suelen aparecer algunos **errores comunes**. Uno de los más frecuentes es confundir **píxeles absolutos** con **posiciones relativas**. Cuando se utilizan valores fijos como `(200,150)`, la ubicación del objeto queda definida únicamente en relación con el origen del sistema de coordenadas `(0,0)`. Esto significa que la posición está fijada respecto al borde superior izquierdo del canvas y no en relación con otros elementos de la escena.

Este tipo de coordenadas puede funcionar en situaciones simples, pero resulta problemático cuando los objetos deberían reaccionar a otros elementos o al comportamiento del usuario. Un ejemplo claro aparece si se piensa en la **mira de un juego de disparos**. La función de la mira podría ser seguir al cursor del mouse, por lo que su posición no debería estar definida por un punto fijo del canvas, sino por la posición actual del mouse.

Si se dibujara una mira utilizando coordenadas fijas como:

```
line(200,150,220,150);  
line(210,140,210,160);
```

la mira quedaría siempre en el mismo lugar de la pantalla, independientemente de cómo se mueva el mouse. En este caso la posición es **absoluta**, porque está anclada a valores específicos de píxeles.

En cambio, si se utiliza la posición del mouse como referencia, la mira puede construirse **de forma relativa** a ese punto. Processing proporciona las variables `mouseX` y `mouseY`, que indican la posición actual del cursor dentro del canvas. Si se toma ese punto como centro de la mira, los elementos que la componen pueden calcularse en relación con esas coordenadas.

Por ejemplo:

```
line(mouseX-10, mouseY, mouseX+10, mouseY);  
line(mouseX, mouseY-10, mouseX, mouseY+10);
```

En este caso, cada línea se dibuja tomando como referencia la posición del mouse. Los valores `-10` y `+10` definen la distancia de los extremos de la mira respecto a ese punto central. La posición de las líneas ya no depende del origen `(0,0)`, sino del lugar donde se encuentra el cursor en ese momento. (Se transportaron al mouse.)

Una forma aún más clara y reutilizable de organizar este tipo de código consiste en **encapsular el dibujo de la mira dentro de una función** que reciba la posición donde debe dibujarse. De esta manera, las coordenadas de referencia, en este caso las centrales, se pasan como parámetros y las líneas de la mira se calculan dentro de la función utilizando posiciones relativas a ese punto.

Por ejemplo:

```
void dibujarMira(float x, float y) {  
  
    line(x-10, y, x+10, y);  
    line(x, y-10, x, y+10);  
  
}
```

Luego, dentro del programa, la función puede llamarse utilizando la posición del mouse:

```
dibujarMira(mouseX, mouseY);
```

El beneficio de esta estrategia es que separa claramente **la definición de la forma** de la mira de **la posición donde se dibuja**. La función describe cómo está construida la mira, mientras que los parámetros `x` e `y` indican dónde ubicarla. Esto permite reutilizar la misma función en distintos contextos: por ejemplo, para dibujar miras en posiciones fijas, seguir distintos objetos en pantalla o incluso representar múltiples miras simultáneamente.

De esta manera, el programa no solo utiliza posiciones relativas, sino que también organiza el código de forma modular, haciendo más fácil modificar, reutilizar y mantener los elementos gráficos dentro del sistema de coordenadas del canvas.

En este ejemplo, la distancia utilizada para construir la mira es el valor `10`, que determina qué tan lejos se extienden las líneas desde el centro. Sin embargo, este valor también podría calcularse mediante una expresión basada en el tamaño del canvas, por ejemplo utilizando `width` o `height`. De esta manera, la escala de la mira también se adaptaría automáticamente al tamaño de la ventana, por ejemplo usando valores como `width/50` o `height/40` en lugar de un número fijo.

Ejercicio integrador.

Como ejercicio de aplicación, se propone realizar una composición simple utilizando figuras básicas —por ejemplo círculos, rectángulos o líneas— ubicadas mediante **posiciones relativas dentro del canvas**. El objetivo es que la organización visual de la composición se mantenga aunque se modifique el tamaño definido en `size()`.

Recuerden además **crear una copia de este programa en su repositorio digital**, organizándolo dentro de la carpeta correspondiente a la materia. Antes de comenzar a programar, utilicen la **carpeta de trabajo para diseñar previamente la composición**, realizando bocetos donde puedan pensar la ubicación relativa de los elementos dentro del canvas. La **carpeta física debe completarse y mantenerse actualizada**, ya que forma parte del proceso de trabajo de la materia y es un requisito importante para su aprobación. En ella deben quedar registrados los bocetos, pruebas de organización espacial y las decisiones que luego se implementarán en el programa.

Para resolver este ejercicio es importante aplicar dos herramientas vistas en clase. Por un lado, el uso de **expresiones** que combinen `width` y `height` para calcular posiciones y tamaños de los elementos (por ejemplo `width/2`, `height*0.3`, `width/5`, etc.). Estas expresiones permitirán definir las coordenadas de manera proporcional al tamaño del canvas en lugar de utilizar valores fijos.

Por otro lado, se sugiere organizar el programa utilizando **funciones o procedimientos** que se encarguen de dibujar algunos de los elementos de la composición. Estas funciones pueden recibir como parámetros las posiciones donde deben dibujarse las figuras, y dentro de ellas calcular las posiciones relativas de sus partes. De esta manera se separa la definición de las formas de la lógica que decide dónde ubicarlas en la pantalla.

Una vez construido el programa, se recomienda **probarlo cambiando las dimensiones del canvas** en la función `size()`. El objetivo es verificar que, gracias al uso de expresiones y funciones, los elementos mantengan su organización espacial y la composición continúe funcionando correctamente en distintos tamaños de ventana.

El **entregable de este ejercicio** consiste en compartir el **link al repositorio o carpeta de Drive** donde estarán organizando y guardando los programas y materiales de trabajo durante el año. Ese espacio funcionará como su **archivo digital de la materia**, por lo que deberá mantenerse actualizado con los ejercicios y versiones de los programas que vayan desarrollando.

Además, deberá incluir **un índice en formato de texto** que permita identificar dónde se encuentra cada trabajo. Este índice puede organizarse **por fecha**, indicando brevemente el contenido o ejercicio correspondiente. El objetivo es que cualquier persona pueda recorrer el repositorio y comprender con facilidad la evolución de los trabajos y localizar rápidamente cada material.

